21/03/2017

EE 241 DIGITAL CIRCUITS
MIDTERM SOLUTION MANUAL

Name:

Q1) (30 points) A combinational circuit is represented by the truth table below. Describe this circuit in Verilog using dataflow modeling. The first output v will be represented in SOP form. The second output w will be represented in POS form.

Inputs			Outputs	
x	y	z	v	w
0	0	0	1	0
0	0	1	0	0
0	1	0	0	0
0	1	1	0	1
1	0	0	0	1
1	0	1	0	0
1	1	0	0	0
1	1	1	1	0

Solution:

Inputs			Output	SOP	Output		POS
x	y	z	v		w	$\overline{w}$	
0	0	0	1	$\bar{x}.\bar{y}.\bar{z}$	0	1	$(x+y+z)$
0	0	1	0		0	1	$(x+y+\bar{z})$
0	1	0	0		0	1	$(x+\bar{y}+z)$
0	1	1	0		1	0	
1	0	0	0		1	0	
1	0	1	0		0	1	$(\bar{x}+y+\bar{z})$
1	1	0	0		0	1	$(\bar{x}+\bar{y}+z)$
1	1	1	1	$x.y.z$	0	1	$(\bar{x}+\bar{y}+\bar{z})$

$v = \bar{x}.\bar{y}.\bar{z} + x.y.z$

$w = (x+y+z).(x+y+\bar{z}).(x+\bar{y}+z).(\bar{x}+y+\bar{z}).(\bar{x}+\bar{y}+z).(\bar{x}+\bar{y}+\bar{z})$

```
----------------------------------------
module question1(v,w,x,y,z);

output v, w;
input x, y, z;

assign v=~x&~y&~z+x&y&z;
assign w=(x|y|z) & (x|y|~z) & (x|~y|z) & (~x|y|~z) & (~x|~y|z) & (~x|~ y|~z);

endmodule
----------------------------------------
```

Q2) (30 points) You are asked to design an arithmetic unit with the following specifications. The unit can only perform addition and subtraction operations on two eight-bit numbers. Only addition operation can perform both addition and subtraction operations. Therefore, remember the two's complement representation.
The arithmetic unit will have its inputs as follows.
num1: first number to be processed which is 8 bits.
num2: second number to be processed which is 8 bits.
op_type: operation type represented in one bit. For addition and subtraction, *op_type* will be 0 and 1 respectively.
The arithmetic unit will have one output as follows.
res: the result of the operation which is 8 bits.

Design your arithmetic unit in Verilog using dataflow modeling.

Solution:

```
----------------------------------------
module arithmetic_unit(res,num1,num2,op_type);

output [7:0] res;
input [7:0] num1, num2;
input op_type;

wire [7:0] add_res, sub_res;

assign add_res=num1+num2;
assign sub_res=num1+(~num2+1);
assign res=(add_res&{8{~op_type}}) + (sub_res&{8{op_type}});

endmodule
----------------------------------------
```

You can use the below testbench file to obtain the results.

```
----------------------------------------
module arithmetic_unit_tb;

reg [7:0] num1,num2;
reg op_type;
wire [7:0] res;

arithmetic_unit UUT(res,num1,num2,op_type);

initial begin
num1 = 8'd12;
num2 = 8'd10;
op_type=1'b1;
#10;
end
endmodule
----------------------------------------
```

Q3) (50 points) We will expand the arithmetic unit in Question 2 by adding four flags representing the result of the operation. These flags are as follows.
N: will be equal to logic level 1 when the result is negative. Otherwise, it will be zero.
C: will be equal to logic level 1 when a carry bit is generated in addition.
V: will be equal to logic level 1 when an overflow occurs in addition or subtraction. Otherwise, it will be zero.
Z: will be equal to logic level 1 when the result is zero. Otherwise, it will be zero.

Modify the arithmetic unit module by expanding it such that you will have four extra output bits as N, C, V, and Z representing the result of the operation.

Solution:

```
----------------------------------------
module arithmetic_unit_modified(res,N,C,V,Z,num1,num2,op_type);

output [7:0] res;
output N,C,V,Z;
input [7:0] num1, num2;
input op_type;

wire [8:0] add_res, sub_res;

assign add_res=num1+num2;
assign sub_res=num1+~num2+1'b1;
assign res=(add_res[7:0]&{8{~op_type}}) + (sub_res[7:0]&{8{op_type}});

//Negative, N
assign N=sub_res[8]&op_type;

//Carry, C
assign C=add_res[8]&~op_type;

//Overflow, V
assign V=sub_res[8]|add_res[8];

//Zero, Z
assign Z=~(res[7]|res[6]|res[5]|res[4]|res[3]|res[2]|res[1]|res[0]);

endmodule
----------------------------------------
```

You can use the below testbench file to obtain the results.

```
----------------------------------------
module arithmetic_unit_modified_tb;

reg [7:0] num1,num2;
reg op_type;
wire [7:0] res;
wire N,C,V,Z;
```

```
arithmetic_unit_modified UUT(res,N,C,V,Z,num1,num2,op_type);

initial begin
num1 = 8'h0D;
num2 = 8'h0F;
op_type=1'b1;

#10;
end

endmodule
-----------------------------------------
```